

Genome assembly cheatsheet

Replace uppercase filenames with your input filenames. OUTPUT names a new output directory unless a tool uses it as a filename prefix. Each command below is one shell line.

1 Assemblers

1.1 SPAdes

Assemble paired-end Illumina reads with the default k-mer sizes

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -t 4 -m 8 -o OUTPUT
```

For these clean synthetic reads, `--only-assembler` skips read error correction. Both mate files must contain matching read pairs in the same order.

Main outputs, inside OUTPUT:

- `contigs.fasta`: assembled sequences; use this first for contig statistics and evaluation.
- `scaffolds.fasta`: contigs joined using linking evidence; unknown stretches appear as N.
- `assembly_graph_with_scaffolds.gfa`: the assembly graph and scaffold paths.

Use one k-mer size for a controlled comparison

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 31 -t 4 -m 8 -o OUTPUT
```

Ignore pairing while keeping exactly the same input bases

```
spades.py --only-assembler --s1 READS_R1.FQ.gz --s2 READS_R2.FQ.gz -k 31 -t 4 -m 8 -o OUTPUT
```

Use a different OUTPUT for each run. Compare contig and scaffold statistics separately.

Increase the k-mer size with paired-end reads

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 127 -t 4 -m 8 -o OUTPUT
```

Repeat the larger k-mer experiment without pairing

```
spades.py --only-assembler --s1 READS_R1.FQ.gz --s2 READS_R2.FQ.gz -k 127 -t 4 -m 8 -o OUTPUT
```

Combine several k-mer sizes

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 21,33,55,77,99,127 -t 4 -m 8 -o OUTPUT
```

1.2 MEGAHIT

Assemble paired-end Illumina reads with the default settings

```
megahit -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -t 4 -m 8589934592 -o OUTPUT
```

Main output: `final.contigs.fa` contains the assembled contigs. Use it for evaluation and graph construction. Here `-m 8589934592` sets an 8 GiB memory budget in bytes.

Choose the k-mer sizes

```
megahit -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz --k-list 21,41,61,81,101,121 -t 4 -m 8589934592 -o OUTPUT
```

Keep only contigs of at least 500 bases

```
megahit -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz --min-contig-len 500 -t 4 -m 8589934592 -o OUTPUT
```

1.3 Flye

Assemble the Oxford Nanopore reads

```
flye --nano-hq NANOPORE.FQ.gz --genome-size 500k --threads 4 --out-dir OUTPUT
```

--nano-hq suits the practical's synthetic Nanopore reads with approximately 1% errors.

Main outputs, inside OUTPUT:

- assembly.fasta: final assembled sequences; use this for evaluation.
- assembly_info.txt: contig length, coverage and circularity information.
- assembly_graph.gfa: the final assembly graph.

Assemble PacBio high-fidelity reads instead

```
flye --pacbio-hifi PACBIO.FQ.gz --genome-size 500k --threads 4 --out-dir OUTPUT
```

1.4 hifiasm

Assemble the PacBio high-fidelity reads

```
hifiasm -o OUTPUT -t 4 -l0 -f0 PACBIO.FQ.gz
```

Here OUTPUT is a filename prefix. For these small haploid genomes, -l0 disables duplicate purging and -f0 disables the Bloom filter.

Main outputs:

- OUTPUT.bp.p_ctg.gfa: primary contig sequences, stored as graph segments; start with this file.
- OUTPUT.bp.r_ctg.gfa: the raw unitig graph, useful for investigating unresolved branches.

Extract primary contig sequences into a FASTA file

```
awk '$1=="S" && $3!="*" {print ">"$2; print $3}' ASSEMBLY.GFA > ASSEMBLY.FA
```

Set ASSEMBLY.GFA to the primary-contig output. ASSEMBLY.FA contains its sequences and can be evaluated with QUAST or Merqury. This extraction does not retain graph links.

Combine PacBio high-fidelity reads with ultralong Oxford Nanopore reads

```
hifiasm -o OUTPUT -t 4 -l0 -f0 --ul NANOPORE.FQ.gz PACBIO.FQ.gz
```

--ul supplies the Nanopore reads; the final input supplies the PacBio high-fidelity reads. Use reads from the same sample and a new output prefix for the hybrid run. Compare its primary contigs with the PacBio-only assembly.

1.5 Verkko

Combine PacBio high-fidelity reads and Oxford Nanopore reads

```
verkko -d OUTPUT --hifi PACBIO.FQ.gz --nano NANOPORE.FQ.gz --no-correction --no-consensus-bam --haploid --local-cpus 4
```

--local-cpus 4 uses four processor threads; add --local-memory 8 to set an 8 GB local memory budget.

The practical uses --no-correction for its already accurate synthetic high-fidelity reads. --no-consensus-bam avoids a known failure in the small-genome recipe. --haploid matches the simulated genomes.

Main outputs, inside OUTPUT:

- assembly.fasta: final assembled sequences; use this first for quality evaluation.
- assembly.homopolymer-compressed.gfa: an assembly graph with homopolymer runs compressed; it is not the final full-length sequence.

2 Read and assembly statistics

2.1 SeqKit

Get basic read or assembly statistics

```
seqkit stats READS.FQ.gz  
seqkit stats ASSEMBLY.FA
```

The terminal output lists sequence count, total bases, and minimum, mean and maximum sequence lengths.

Include N50 and additional length and quality statistics

```
seqkit stats -a READS_R1.FQ.gz READS_R2.FQ.gz PACBIO.FQ.gz NANOPORE.FQ.gz  
seqkit stats -a ASSEMBLY.FA
```

N50 is the sequence length at which sequences of that length or longer account for at least half of all assembled bases. For paired-end coverage, count the bases in both mate files.

Compare assembly sequences of at least 500 bases

```
seqkit seq -m 500 ASSEMBLY.FA | seqkit stats -a
```

This filters complete FASTA records by length. It does not split scaffolds at unknown bases, so use contig FASTA files for a contig comparison.

Write a statistics table to a file

```
seqkit stats -a -T ASSEMBLY_1.FA ASSEMBLY_2.FA > STATISTICS.tsv
```

STATISTICS.tsv is a tab-separated table that can be opened in a spreadsheet. Compare sequence count, total bases, N50 and longest sequence alongside correctness metrics.

3 Assembly evaluation

3.1 QUAST

Evaluate one A assembly against the A reference

```
quast.py ASSEMBLY.FA -r REFERENCE_A.FA --min-contig 500 --threads 4 -o OUTPUT
```

REFERENCE_A.FA is the supplied true genome for sample A. Use it to evaluate A assemblies after assembly. It is not the true genome for B.

Main outputs, inside OUTPUT:

- `report.html`: the main interactive report; open this file in a web browser first.
- `icarus.html`: contig alignment viewer; inspect missing regions, repeated sequence and suspicious joins.
- `report.tsv`: tab-separated metrics for your worksheet or spreadsheet.
- `report.pdf`: printable summary and plots.

Compare several assemblies of A in the same report

```
quast.py ASSEMBLY_1.FA ASSEMBLY_2.FA ASSEMBLY_3.FA -r REFERENCE_A.FA --min-contig 500 --threads 4 -o OUTPUT
```

Metrics to inspect:

- **Genome fraction**: how much of the reference is covered by assembly alignments.
- **Duplication ratio**: aligned assembly bases relative to covered reference bases; excess sequence can indicate redundancy.
- **Misassemblies**: inconsistencies in aligned contig structure; inspect the alignment evidence.
- **NGA50**: aligned-block contiguity, using reference length and splitting blocks at extensive misassemblies.
- **Mismatches and indels per 100 kb**: nucleotide disagreement with the reference.

The reference is circular but stored as a linear FASTA. Keep QUAST's default circular-origin handling; do not add `--fragmented` for this practical.

3.2 Meryl

Count k-mers from both Illumina mate files

```
meryl count k=21 threads=4 memory=2 READS_R1.FQ.gz READS_R2.FQ.gz output READS.meryl
```

Use reads from the same sample as the assembly. `READS.meryl` is a database directory containing k-mer counts, not a FASTA file. Keep the `.meryl` suffix for Merqury. Pairing information is not used in this count.

Inspect the read k-mer multiplicity distribution

```
meryl histogram READS.meryl > READS.hist
```

`READS.hist` lists multiplicity and the number of distinct k-mers at each multiplicity. Low-count k-mers often include sequencing errors; repeated sequence can form higher-multiplicity peaks.

3.3 Merqury

Evaluate one assembly without a reference genome

```
OMP_NUM_THREADS=4 merqury.sh READS.meryl ASSEMBLY.FA EVALUATION
```

Run in a fresh working directory with the read database and assembly accessible. The supplied environment sets `MERQURY`. `EVALUATION` is an output filename prefix.

Main outputs:

- `EVALUATION.qv`: estimated consensus quality and assembly-only k-mer counts; inspect this first.
- `EVALUATION.completeness.stats`: recovery of reliable read k-mers; this is not the percentage of genome bases recovered.
- `EVALUATION.*.spectra-cn.*.png`: plots relating read multiplicity to assembly copy number.
- `EVALUATION.spectra-asm.*.png`: plots separating read-supported and assembly-only k-mers.

Compare another assembly using the same read evidence

```
OMP_NUM_THREADS=4 merqury.sh READS.meryl ASSEMBLY_2.FA EVALUATION_2
```

Run the second command in another fresh working directory. Use A reads for A assemblies and B reads for B assemblies. Do not treat A and B as two haplotypes of one individual.

Infinite estimated quality means no unsupported assembly k-mers were observed. It does not prove that every base, repeat copy or structural join is correct. Read coverage holes can also hide missing sequence or penalise sequence recovered by another technology.

4 Graph construction and exploration

4.1 GGCAT

Build a linked graph from one assembly

```
ggcat build -k 31 -s 1 -e -j 4 -m 1 ASSEMBLY.FA -o GRAPH.FA
```

GRAPH.FA stores graph-node sequences and links in BCALM-style FASTA, ready for Vizitig. -e retains links. For assembly inputs, -s 1 retains sequence that occurs only once.

Build a graph from Illumina reads

```
ggcat build -k 31 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o GRAPH.FA
```

-s 3 filters low-count read k-mers. This removes many error k-mers but can also remove true low-depth sequence. GGCAT does not use paired-end links here.

Increase the k-mer size

```
ggcat build -k 63 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o GRAPH.FA  
ggcat build -k 127 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o GRAPH.FA
```

Choose a different output filename for each graph. Compare branching and repeat resolution, rather than matching node identifiers between graphs.

Build the sequence union of two assemblies

```
ggcat build -k 31 -s 1 -e -j 4 -m 1 ASSEMBLY_A.FA ASSEMBLY_B.FA -o GRAPH.FA
```

Use the assembly FASTAs as inputs. This identifies shared sequence at the chosen k-mer size; appending native assembly graph files would not do that.

4.2 Vizitig

Import, index and display a GGCAT graph

```
vizitig build GRAPH.FA -k 31 -n GRAPH_NAME --edge-annotation  
vizitig index build GRAPH_NAME  
vizitig run
```

Match `-k` to the value used by GGCAT. `GRAPH_NAME` names the stored graph inside Vizitig; choose a new name for each import. The imported graph and index are kept in Vizitig's configured data store, rather than written as an assembly FASTA.

Open the interface: leave the server running in the terminal. On the same computer, open a web browser and enter `http://localhost:4242` in the address bar. This is the address of your local Vizitig server.

Load the graph: select `GRAPH_NAME` in the graph selector. Enter `All` in the query box, then click **Fetch nodes**. The graph and its node table will appear.

Add sample colours to the two-assembly union

```
vizitig color GRAPH_NAME -f ASSEMBLY_A.FA -m A -c '#0072B2'  
vizitig color GRAPH_NAME -f ASSEMBLY_B.FA -m B -c '#D55E00'
```

These commands label nodes according to sequence membership in A and B. In the interface, choose **Autocolor** → **Color** → **BandHighlight** → **Apply** to display the labels. Shared nodes may carry both labels.

Run Vizitig inside the Docker container

```
vizitig run --hostname 0.0.0.0 --no-launch-browser
```

Publish container port 4242 when starting Docker, as shown in the setup guide. Open `http://localhost:4242` in the host computer's browser. For a remote teaching server, use the setup guide's secure-shell tunnel.

Find shared sequence: enter this in the query box, then click **Fetch nodes**

Color(A) AND Color(B)

Find sequence present only in A

Color(A) AND NOT Color(B)

Find sequence present only in B

Color(B) AND NOT Color(A)

Inspect a branch: select a node and use **Expand** to fetch its neighbours. **Center**, **Linearize** and **Orient** help arrange the local graph. Inspect node sequences and lengths in the table.

Compare a single-nucleotide variant: find two short bubble arms between shared flanks. Select their nodes in the table and use **Open aligner** to compare the sequences.

Investigate structural differences: inspect shared flanks, alternative paths, junctions and orientation. A sample-specific path may also reflect missing coverage or an assembly error. Colours alone do not establish repeat copy number.

Keep the evidence: name and **Save** a useful graph view. Record node identifiers, sequences, assembly source and your interpretation in the worksheet.

5 Practical speedrun

5.1 Assemble

Run this sequence for A, then B, using that sample's reads and new output names. Uppercase names are placeholders. Start in the supplied software environment.

Check the reads

```
seqkit stats -a READS_R1.FQ.gz READS_R2.FQ.gz PACBIO.FQ.gz NANOPORE.FQ.gz
```

Compare pairing and k-mer size with SPAdes

```
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 31 -t 4 -m 8 -o PAIRED_31
spades.py --only-assembler --s1 READS_R1.FQ.gz --s2 READS_R2.FQ.gz -k 31 -t 4 -m 8 -o UNPAIRED_31
spades.py --only-assembler -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -k 127 -t 4 -m 8 -o PAIRED_127
spades.py --only-assembler --s1 READS_R1.FQ.gz --s2 READS_R2.FQ.gz -k 127 -t 4 -m 8 -o UNPAIRED_127
```

Compare contigs.fasta first, then scaffolds.fasta, from each output. Check whether pairing and larger k-mers improve contiguity.

Compare short-read assemblers

```
megahit -1 READS_R1.FQ.gz -2 READS_R2.FQ.gz -t 4 -m 8589934592 -o MEGAHIT_OUTPUT
```

Use final.contigs.fa for the comparison.

Assemble long reads with Flye and hifiasm

```
flye --nano-hq NANOPORE.FQ.gz --genome-size 500k --threads 4 --out-dir FLYE_NANOPORE
flye --pacbio-hifi PACBIO.FQ.gz --genome-size 500k --threads 4 --out-dir FLYE_PACBIO
hifiasm -o HIFIASM -t 4 -l0 -f0 PACBIO.FQ.gz
awk '$1=="S" && $3!="*" {print ">"$2; print $3}' HIFIASM.bp.p_ctg.gfa > HIFIASM.FA
```

Use Flye's assembly.fasta and assembly_info.txt, then HIFIASM.FA. Compare contiguity and inspect circularity in Flye's table.

5.2 Evaluate

Compare assembly lengths and N50

```
seqkit stats -a ASSEMBLY_1.FA ASSEMBLY_2.FA ASSEMBLY_3.FA
```

Repeat for the remaining assemblies. Compare contigs of at least 500 bases consistently; use `seqkit seq -m 500 ASSEMBLY.FA | seqkit stats -a` for a filtered check.

Evaluate A assemblies against the supplied A reference

```
quast.py ASSEMBLY_1.FA ASSEMBLY_2.FA ASSEMBLY_3.FA -r REFERENCE_A.FA --min-contig 500 --threads 4 -o QUAST_A
```

Open `report.html` first, then `icarus.html`. Compare genome fraction, duplication, misassemblies and NGA50. Do not use the A reference as ground truth for B.

Build read evidence for reference-free evaluation

```
meryl count k=21 threads=4 memory=2 READS_R1.FQ.gz READS_R2.FQ.gz output READS.meryl  
meryl histogram READS.meryl > READS.hist
```

Build one database per sample. Use the corresponding sample's Illumina reads to evaluate every assembly of that sample.

Run Merqury on each assembly

```
OMP_NUM_THREADS=4 merqury.sh READS.meryl ASSEMBLY.FA EVALUATION
```

Run each assembly in a fresh working directory, with the input database and assembly accessible there. Inspect `EVALUATION.qv`, `EVALUATION.completeness.stats` and the copy-number spectra plots. Record quality and completeness alongside N50.

5.3 Optional hybrid assemblies

Combine PacBio and Nanopore reads

```
hifiasm -o HYBRID -t 4 -l0 -f0 --ul NANOPORE.FQ.gz PACBIO.FQ.gz  
awk '$1=="S" && $3!="*" {print ">"$2; print $3}' HYBRID.bp.p_ctg.gfa > HYBRID.FA  
verkko -d VERKKO_OUTPUT --hifi PACBIO.FQ.gz --nano NANOPORE.FQ.gz --no-correction --no-consensus-bam --haploid
```

Evaluate `HYBRID.FA` and Verkko's assembly.fasta in the same way. Compare with the assemblies from each technology alone.

5.4 Explore graphs and compare genomes

Build Illumina graphs at three k-mer sizes

```
gcat build -k 31 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o READ_GRAPH_31.FA
gcat build -k 63 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o READ_GRAPH_63.FA
gcat build -k 127 -s 3 -e -j 4 -m 1 READS_R1.FQ.gz READS_R2.FQ.gz -o READ_GRAPH_127.FA
```

Import the read graphs and build an assembly graph

```
vizitig build READ_GRAPH_31.FA -k 31 -n READ_31 --edge-annotation
vizitig index build READ_31
vizitig build READ_GRAPH_63.FA -k 63 -n READ_63 --edge-annotation
vizitig index build READ_63
vizitig build READ_GRAPH_127.FA -k 127 -n READ_127 --edge-annotation
vizitig index build READ_127
gcat build -k 31 -s 1 -e -j 4 -m 1 ASSEMBLY.FA -o ASSEMBLY_GRAPH.FA
vizitig build ASSEMBLY_GRAPH.FA -k 31 -n ASSEMBLY_GRAPH --edge-annotation
vizitig index build ASSEMBLY_GRAPH
```

Build, index and colour the two-assembly union

```
gcat build -k 31 -s 1 -e -j 4 -m 1 ASSEMBLY_A.FA ASSEMBLY_B.FA -o UNION.FA
vizitig build UNION.FA -k 31 -n UNION --edge-annotation
vizitig index build UNION
vizitig color UNION -f ASSEMBLY_A.FA -m A -c '#0072B2'
vizitig color UNION -f ASSEMBLY_B.FA -m B -c '#D55E00'
vizitig run --hostname 0.0.0.0 --no-launch-browser
```

For ASSEMBLY_A.FA and ASSEMBLY_B.FA, use your selected assemblies or the supplied mystery-comparison assemblies. Vizitig graph names must be unused in your data store.

In the browser: open <http://localhost:4242> on the computer running the server. Docker requires port 4242 to be published. Leave the terminal running.

- Select each read graph; query All and click **Fetch nodes**. Compare branches and repeat resolution across k-mer sizes.
- Select ASSEMBLY_GRAPH and inspect unresolved branches. Select UNION; fetch All. Choose **Autocolor** → **Color** → **BandHighlight** → **Apply**.
- Query Color(A) AND NOT Color(B) or Color(B) AND NOT Color(A). Expand neighbours to find shared flanks.
- Compare bubble-arm sequences with **Open aligner**; investigate a candidate single-nucleotide variant and a structural difference.
- Save a graph view and record node identifiers, assembly sources and evidence. Consider missing coverage and assembly errors before calling a variant.